

CS-499 Milestone Two: Enhancement Narrative

Enhancement One: Software Design and Engineering

Michael King

7/17/2026

Artifact Description

The artifact that will be used for the Software Design and Engineering enhancement will be the 3D computational graphics scene that was originally developed for CS-330: Computational Graphics and Visualization. The project was developed as part of the CS-330 course requirements and is a complete interactive 3D still life of a kitchen constructed in C++ using the OpenGL 3.3 Core Profile graphics API together with the windowing and extension, loading the GLFW and GLEW libraries. The scene renders seven independent objects: a wooden table surface, a yellow cutting board, a ceramic coffee mug, a fabric mug handle, a terra cotta ramekin bowl, a stainless steel fork, and a stainless steel knife, all with per-object Phong shading, texture mapping, and a two-light illumination model. The original application also employs a free-flying FPS style camera with keyboard and mouse input, perspective and orthographic projection, and UV tiling. The code is organized into six source files: MainCode.cpp, ViewManager, ShaderManager, SceneManager, ShapeMeshes, and camera.h.

Why This Artifact Was Selected

The artifact was chosen as a representative sample of my portfolio because it is the most technically rich and visually impressive project I have completed during my Computer Science studies. In particular, performing graphics programming on the computer calls for a programmer

to be familiar with low-level systems programming, linear algebra, the rendering pipeline of the GPU, and real-time user inputs, something that is rarely captured in one project. The decision to port a project from C++ to Python was a large step in its own right; it was not simply based on superficial concerns like language syntax, but represented a complete revisit of the entire architecture of the original implementation. The artifact is visually interesting and would be a strong visual component of a professional ePortfolio.

Components That Showcase Software Engineering Skills

The enhancement shows skills in several areas of software engineering. Firstly, it involved **cross-language porting** across six interdependent modules, requiring knowledge of the C++ and Python semantics, memory models, and idioms. For instance, the C++ *camera.h* header was ported with the GLM vector/matrix mathematics library; the Python version used NumPy arrays with the Pyrr matrix library to produce identical behavior. Secondly, **object-oriented design** was contained: all C++ class structures were ported to Python classes with documented separation of public and private classes and methods and inline docstrings that enhanced readability beyond the C++ implementations. Thirdly, the **graphics pipeline infrastructure** was developed from scratch in Python using GLSL shaders with the ShaderManager, generating VAO and VBO buffers with ShapeMeshes, uploading Phong light sources with SceneManager, and building a projection matrix with ViewManager. Fourth, **procedural mesh generation** was used in place of the original hand-written vertex arrays for the curved surfaces. The original C++ ShapeMeshes.cpp contained encoded arrays for the series of cylinders, tori, and tapered cylinders made of thousands of lexical floating-point values. In the Python port, these same shapes were needed in the same resolution, so the algorithm was replaced to generate the shapes from the number of sectors as required parameters, and it produces identical geometry. Fifth, **substitution**

of libraries was also made with some care: Pygame CE replaced the GLFW library, the Pillow library replaced stb_image.h, and Pyrr and NumPy replaced GLM, again, all while keeping the same mathematics and exact scene appearance.

How the Artifact was Improved

The artifact was improved in a number of tangible ways. The Python port alone adds inline docstrings to every class and method, complete with parameter and return annotations and insightful commentary on design decisions, most of which were absent in the C++ version. The procedural approach to mesh generation cuts down the 96KB of raw vertex data that made up the ShapeMeshes (one of the larger files in the original project) and packs it into a manageable, parameter-driven algorithm. The class hierarchy is more modular due to Python's module archiving (rather than larger header/implementation file pairs common in C++), and each class is entirely self-sufficient. Input handling is simpler due to Pygame's lack of callback architecture (replacing the key event handler callbacks from GLFW), and key debouncing is achieved through clear logical conditionals instead of raising state variables. A shader load failure check is added to the application entry point to prevent the program from receiving an invalid ID if the shader cannot be compiled, to prevent silent continuation; instead, a clean exit is given. The call to enable the depth test was moved out of the render method (to once a second, previously 60 times a second). The return values from glGenTextures/glGenVertexArrays are now plain Python integers, passed through the Python int function, rather than handling a behavior change in PyOpenGL 3.14, where these functions now return arrays rather than scalar integer types.

Course Outcome Alignment

The course outcomes planned in Module One were supported by this enhancement. The first planned outcome is fulfilled here through the cross-language, cross-toolchain migration of a working OpenGL program, by utilizing the standard Python libraries used in the scientific and visualization computing industries, and by the described architectural enhancements. The ability to migrate a complex graphics program from one language and set of libraries to another is an industry-related competency in the field of game development, simulation, and scientific visualization.

The algorithmic principles and computer science practices addressed in the second planned outcome were well-thought-out during the trade-off analysis, such as whether to use procedural generation of the mesh instead of literal vertex arrays (which trades file size and generation time for readability and parameterizability), and whether to use Pygame CE instead of Pygame (which trades the expected Pygame library for support in the latest Python 3.14). These decisions were planned, documented, and supported using software engineering logic. No modifications to the existing outcome or coverage plan were necessary. The improvement made is the same as the plan outlined in the first module, and all six planned Python modules have been written and verified to run successfully.

What Was Learned

Of the major lessons learned during this enhancement, the one that stands out most is that there are nuanced differences between C++ and Python in language semantics that can make software behave in ways that are difficult to predict. In particular, the subtle “idiomatic” behavior of the input event handling system led to the most educational bug, where the camera’s movement speed would get set back to an unusually low value each time any movement key was

pressed. After exhaustive debugging of the input event handling code, this was narrowed down to a one-character typo in *camera.py*:

```
velocity = self.movement_speed = delta_time
```

I used Python's chained assignment operator to apply `delta_time` to both `velocity` and `self.movement_speed` at once. In C++, this expression chains assignments from right to left and leaves the two variables as separate memory locations, resulting in the desired behavior. In Python, the chained assignment assigns the same value to each target, causing `movement_speed` to be reset to the frame delta time for each key press. The proper line is

```
velocity = self.movement_speed * delta_time
```

This reinforced the learning that cross-language porting is not simply a matter of syntactic translation, but actually involves reasoning about the semantic differences of language constructs that are visually similar. It also made evident that a systematic debugging approach can be powerful: The symptom of a speed reset on movement was quite distinct and easily generalized to the keyboard handler; the only place in *camera.py* where `movement_speed` could be set on keypress was the `process_keyboard` method.

A second important learning point was the return types of OpenGL functions when called via PyOpenGL on Python 3.14. With newer versions of PyOpenGL, the functions `glGenTextures(1)`, `glGenVertexArrays(1)`, and `glGenBuffers(1)` now return a NumPy array or list type instead of a single return value for the handle (as returned by the C++ call). Passing such an object directly to `glBindTexture` or `glBindVertexArray` results in a `TypeError`. An understanding of the behavior of the PyOpenGL source, combined with an understanding of Python's typeflow,

allowed for a strong `GenericInt()` normalization pattern to solve this issue by always producing a simple integer regardless of an iterable value being thrown at it.

Challenges Faced

One major challenge was the coordination system and matrix convention difference between GLM (used in the C++ original) and Pyrr (used in the Python port). SQLite GLM is ordered in column-major order, with the application of column vectors, whereas the Pyrr is ordered in row-major order, with the application of row vectors. By way of illustration, a matrix composition that is applied left to right in GLM ($T * R * S$ becomes translate, then rotate, then scale) has to be written left to right in Pyrr to get the same visual output, because Pyrr's transposed storage order turns the NumPy dot product in such a way that application order appears to be reversed with respect to that of GLM. To check that this was correct, several manual matrix calculations with specific numbers were tested, comparing the screen output of the listed scene to the intended one.

The secondary challenge was finding a stable camera speed control for different input hardware configurations. The C++ used the GLFW scroll callback, which normalized the delta value. Pygame-CE on Windows produces `MOUSEWHEEL` events whose delta varies between hardware scroll wheels and precision touchpads, and, on some platforms, false OS-level scroll events when moving the keyboard camera. Many attempts were made to resolve this (noise, touch, event-filtering, time, debouncing, movement key guards) until it was found that its root cause was, in fact, the chained assignment bug, which fixed the primary symptom. That scroll speed control was temporarily replaced with the [and] keyboard shortcuts.

The third challenge was support for Python 3.14. Pygame itself (through 2.6.x versions) does not officially provide pre-built wheel distributions for this version. As a result, I looked for and used Pygame-CE as an appropriate wheel replacement. This process involved examining Python package wheel options and making strategic choices for 3.14 Pygame to identify a suitable substitute, a scenario often encountered when evaluating real-world dependencies in professional software development environments.

Conclusion

The CS-330 3D scene Python port is a tangible, intellectually rigorous step forward for the ePortfolio. It shows the ability to take a difficult piece of existing code, port it through a new language and different toolchains, re-architect and thoroughly document it, and fix painful language-inherent bugs. It was a true learning experience, and it has substantial overlap with real-world software engineering: the semantics of Python's chained assignment operator, the use of PyOpenGL type names in Python 3.14, and the ins and outs of cross-platform input management.